

实验 4：言论过滤器与集成学习分类算法实现及对比分析

一：实验目的

- 1、掌握朴素贝叶斯程序实现方法；
- 2、掌握朴素贝叶斯算法的程序含义；
- 3、了解集成学习中 AdaBoost 算法的概念；
- 4、掌握 AdaBoost 算法的实现流程；
- 5、理解朴素贝叶斯、AdaBoost 算法的一般应用。

二：实验环境

- 1、系统：windows 7、windows 10 或 windows 11
- 2、软件：PyCharm

三：实验内容

（一）言论过滤器

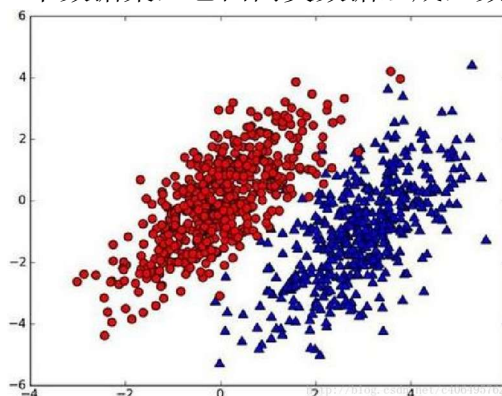
朴素贝叶斯算法是有监督的学习算法，解决的是分类问题，如客户是否流失、是否值得投资、信用等级评定等多分类问题。算法的优点在于简单易懂、学习效率高、在某些领域的分类问题中能够与决策树、神经网络相媲美。但由于该算法以自变量之间的独立（条件特征独立）性和连续变量的正态性假设为前提，就会导致算法精度在某种程度上受影响。

1、朴素贝叶斯理论

朴素贝叶斯是贝叶斯决策理论的一部分，所以在讲述朴素贝叶斯之前有必要快速了解一下贝叶斯决策理论。

（1）贝叶斯决策理论

假设现在我们有一个数据集，它由两类数据组成，数据分布如下图所示：



现在用 $P_1(x,y)$ 表示数据点 (x,y) 属于类别 1 (图中红色圆点表示的类别) 的概率, 用 $P_2(x,y)$ 表示数据点 (x,y) 属于类别 2 (图中蓝色三角形表示的类别) 的概率, 那么对于一个新数据点 (x,y) , 可以用下面的规则来判断它的类别:

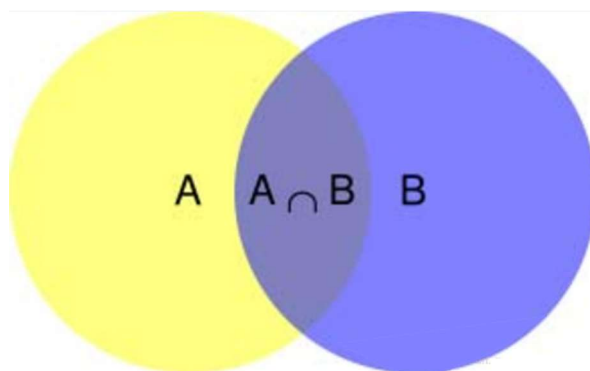
如果 $P_1(x,y) > P_2(x,y)$, 那么类别为 1

如果 $P_1(x,y) < P_2(x,y)$, 那么类别为 2

也就是说, 我们会选择高概率对应的类别。这就是贝叶斯决策理论的核心思想, 即选择具有最高概率的决策。已经了解了贝叶斯决策理论的核心思想, 那么接下来, 就是学习如何计算 P_1 和 P_2 概率。

(2) 条件概率

在学习计算 P_1 和 P_2 概率之前, 我们需要了解什么是条件概率(Conditional probability), 就是指在事件 B 发生的情况下, 事件 A 发生的概率, 用 $P(A|B)$ 来表示。



根据上图, 可以很清楚地看到在事件 B 发生的情况下, 事件 A 发生的概率就是 $P(A \cap B)$ 除以 $P(B)$ 。

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

因此,

$$P(A \cap B) = P(A|B)P(B)$$

同理可得,

$$P(A \cap B) = P(B|A)P(A)$$

所以,

$$P(A|B)P(B) = P(B|A)P(A)$$

即

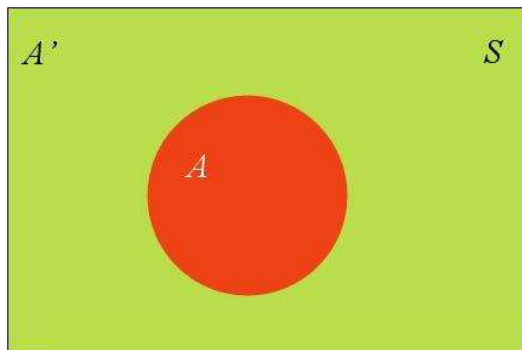
$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

这就是条件概率的计算公式。

(3) 全概率公式

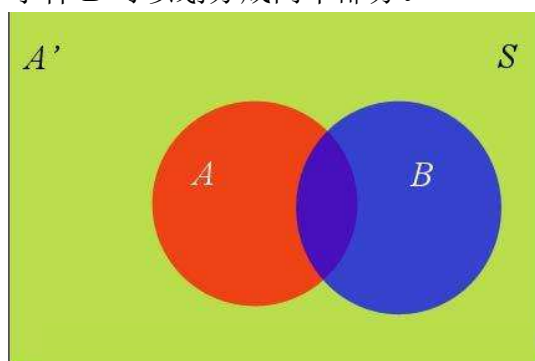
除了条件概率以外，在计算 P_1 和 P_2 的时候，还要用到全概率公式，因此，这里继续推导全概率公式。

假定样本空间 S 是两个事件 A 与 A' 的和，所以有如下关系



上图中，红色部分是事件 A ，绿色部分是事件 A' ，它们共同构成了样本空间 S 。

在这种情况下，事件 B 可以划分成两个部分。



即

$$P(B) = P(B \cap A) + P(B \cap A')$$

在上一节的推导当中，我们已知

$$P(B \cap A) = P(B | A)P(A)$$

所以，

$$P(B) = P(B | A)P(A) + P(B | A')P(A')$$

这就是全概率公式。它的含义是，如果 A 和 A' 构成样本空间的一个划分，那么事件 B 的概率，就等于 A 和 A' 的概率分别乘以 B 对这两个事件的条件概率之和。

将这个公式代入上一节的条件概率公式，就得到了条件概率的另一种写法：

$$P(A | B) = \frac{P(B | A)P(A)}{P(B | A)P(A) + P(B | A')P(A')}$$

(4) 贝叶斯推断

对条件概率公式进行变形，可以得到如下形式：

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

我们把 $P(A)$ 称为"先验概率" (Prior probability)，即在 B 事件发生之前，我们对 A 事件概率的一个判断。

$P(A|B)$ 称为"后验概率" (Posterior probability)，即在 B 事件发生之后，我们对 A 事件概率的重新评估。

$P(B|A)/P(B)$ 称为"可能性函数" (Likelihood)，这是一个调整因子，使得预估概率更接近真实概率。

所以，条件概率可以理解成下面的式子：

后验概率 = 先验概率 x 调整因子

这就是贝叶斯推断的含义。我们先预估一个"先验概率"，然后加入实验结果，看这个实验到底是增强还是削弱了"先验概率"，由此得到更接近事实的"后验概率"。

在这里，如果"可能性函数" $P(B|A)/P(B) > 1$ ，意味着"先验概率"被增强，事件 A 的發生的可能性变大；如果"可能性函数" $= 1$ ，意味着 B 事件无助于判断事件 A 的可能性；如果"可能性函数" < 1 ，意味着"先验概率"被削弱，事件 A 的可能性变小。

(5) 朴素贝叶斯推断

理解了贝叶斯推断，那么让我们继续看看朴素贝叶斯。贝叶斯和朴素贝叶斯的概念是不同的，区别就在于“朴素”二字，朴素贝叶斯对条件个概率分布做了条件独立性的假设。比如下面的公式，假设有 n 个特征：

$$P(a|X) = P(X|a)P(a) = P(x_1, x_2, x_3, \dots, x_n|a)P(a)$$

由于每个特征都是独立的，我们可以进一步拆分公式

$$\begin{aligned} P(a|X) &= P(X|a)P(a) \\ &= \{P(x_1|a) \times P(x_2|a) \times P(x_3|a) \times \dots \times P(x_n|a)\} P(a) \end{aligned}$$

这样我们就可以进行计算了。如果有些迷糊，让我们从一个例子开始讲起，你会看到贝叶斯分类器很好懂，一点都不难。

某个医院早上来了六个门诊的病人，他们的情况如下表所示：

症状	职业	疾病
打喷嚏	护士	感冒

症状	职业	疾病
打喷嚏	农夫	过敏
头痛	建筑工人	脑震荡
头痛	建筑工人	感冒
打喷嚏	教师	感冒
头痛	教师	脑震荡

现在又来了第七个病人，是一个打喷嚏的建筑工人。请问他患上感冒的概率有多大？

根据贝叶斯定理：

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

可得：

$$P(\text{感冒} | \text{打喷嚏} \times \text{建筑工人}) = \frac{P(\text{打喷嚏} \times \text{建筑工人} | \text{感冒}) \times P(\text{感冒})}{P(\text{打喷嚏} \times \text{建筑工人})}$$

根据朴素贝叶斯条件独立性的假设可知，“打喷嚏”和“建筑工人”这两个特征是独立的，因此，上面的等式就变成了

$$P(\text{感冒} | \text{打喷嚏} \times \text{建筑工人}) = \frac{P(\text{打喷嚏} | \text{感冒}) \times P(\text{建筑工人} | \text{感冒}) \times P(\text{感冒})}{P(\text{打喷嚏}) \times P(\text{建筑工人})}$$

这里可以计算：

$$P(\text{感冒} | \text{打喷嚏} \times \text{建筑工人}) = \frac{0.66 \times 0.33 \times 0.5}{0.5 \times 0.33} = 0.66$$

因此，这个打喷嚏的建筑工人，有 66% 的概率是得了感冒。同理，可以计算这个病人患上过敏或脑震荡的概率。比较这几个概率，就可以知道他最可能得什么病。

这就是贝叶斯分类器的基本方法：在统计资料的基础上，依据某些特征，计算各个类别的概率，从而实现分类。

同样，在编程的时候，如果不需要求出所属类别的具体概率， $P(\text{打喷嚏}) = 0.5$ 和 $P(\text{建筑工人}) = 0.33$ 的概率可以不用求。

下面以文本分类为例，进行实践编程。

2、文本分类

以在线社区留言为例。为了不影响社区的发展，我们要屏蔽侮辱性的言论，所以要构建一个快速过滤器，如果某条留言使用了负面或者侮辱性的语言，那么就将该留言标志为内容不当。过滤这类内容是一个很常见的需求。对此问题建立两个类型：侮辱类和非侮辱类，使用 1 和 0 分别表示。

我们把文本看成单词向量或者词条向量，也就是说将句子转换为向量。考虑出现所有文档中的单词，再决定将哪些单词纳入词汇表或者说所要的词汇集合，然后必须要将每一篇文档转换为词汇表上的向量。简单起见，我们先假设已经将本文切分完毕，存放到列表中，并对词汇向量进行分类标注，数据如下：

	文档 ID	文档中的词	文本分类
训练集	1	'my', 'dog', 'has', 'flea', 'problems', 'help', 'please'	0
	2	'maybe', 'not', 'take', 'him', 'to', 'dog', 'park', 'stupid'	1
	3	'my', 'dalmation', 'is', 'so', 'cute', 'I', 'love', 'him'	0
	4	'stop', 'posting', 'stupid', 'worthless', 'garbage'	1
	5	'mr', 'licks', 'ate', 'my', 'steak', 'how', 'to', 'stop', 'him'	0
	6	'quit', 'buying', 'worthless', 'dog', 'food', 'stupid'	1
测试集	1	'love', 'my', 'dalmation'	?
	2	'stupid', 'garbage'	?

判断测试数据分别属于哪一类文本？

我们已经将 `postingList` 是存放词条列表中，`classVec` 是存放每个词条的所属类别，1 代表侮辱类，0 代表非侮辱类。

```
249
250 ▼ if __name__ == '__main__':
251     postingList, classVec = loadDataSet()
252     for each in postingList:
253         print(each)
254     print(classVec)
255

['my', 'dog', 'has', 'flea', 'problems', 'help', 'please']
['maybe', 'not', 'take', 'him', 'to', 'dog', 'park', 'stupid']
['my', 'dalmation', 'is', 'so', 'cute', 'I', 'love', 'him']
['stop', 'posting', 'stupid', 'worthless', 'garbage']
['mr', 'licks', 'ate', 'my', 'steak', 'how', 'to', 'stop', 'him']
['quit', 'buying', 'worthless', 'dog', 'food', 'stupid']
[0, 1, 0, 1, 0, 1]
[Finished in 0.5s]
```

继续编写代码，前面我们已经说过我们要先创建一个词汇表，并将切分好的词条转换为词条向量。

从下面运行结果可以看出，`postingList` 是原始的词条列表，`myVocabList` 是词汇表。`myVocabList` 是所有单词出现的集合，没有重复的元素，它是用来将词条向量化的，一个单词在词汇表中出现过一次，那么就在相应位置记作 1，如果

没有出现就在相应位置记作 0。`trainMat` 是所有的词条向量组成的列表。它里面存放的是根据 `myVocabList` 向量化的词条向量。

我们已经得到了词条向量。接下来，我们就可以通过词条向量训练朴素贝叶斯分类器。

```

247
248
249
250
251
252
253
254
255
256
257
258
259

```

```

if __name__ == '__main__':
    postingList, classVec = loadDataSet()
    myVocabList = createVocabList(postingList)
    print('myVocabList:\n', myVocabList)
    trainMat = []
    for postinDoc in postingList:
        trainMat.append(setOfWords2Vec(myVocabList, postinDoc))
    p0V, p1V, pAb = trainNB0(trainMat, classVec)
    print('p0V:\n', p0V)
    print('p1V:\n', p1V)
    print('classVec:\n', classVec)
    print('pAb:\n', pAb)

```

```

myVocabList:
['him', 'is', 'not', 'cute', 'love', 'posting', 'garbage', 'flea', 'quit', 'take', 'stop', 'help', 'buying', 'has', 'licks', 'please', 'my', 'dalmation',
'worthless', 'problems', 'to', 'maybe', 'mr', 'park', 'how', 'so', 'stupid', 'food', 'steak', 'ate', 'I', 'dog']

```

```

p0V:
[ 0.08333333  0.04166667  0.          0.04166667  0.04166667  0.          0.          0.08333333
 0.04166667  0.          0.          0.04166667  0.04166667  0.          0.          0.04166667
 0.04166667  0.04166667  0.04166667  0.125          0.04166667  0.          0.04166667
 0.04166667  0.04166667  0.          0.04166667  0.04166667  0.04166667  0.04166667
 0.04166667]

```

p0V的倒数第6个概率

```

p1V:
[ 0.05263158  0.          0.05263158  0.          0.          0.05263158  0.05263158  0.05263158
 0.05263158  0.          0.          0.          0.          0.          0.          0.05263158
 0.05263158  0.05263158  0.05263158  0.          0.          0.05263158  0.05263158  0.05263158
 0.          0.          0.15789474  0.05263158  0.          0.          0.          0.05263158]

```

p1V的倒数第6个概率

```

classVec:
[[0, 1, 0, 1, 0, 1]]
pAb:
[0.5]
[Finished in 0.3s]

```

classifyNB()函数，用朴素贝叶斯公式，计算词条向量属于侮辱类和非侮辱类的概率。运行结果如下：

```
171 if __name__ == '__main__':
172     testingNB()

p0: 0.0
p1: 0.0
['love', 'my', 'dalmation'] 属于非侮辱类
p0: 0.0
p1: 0.0
['stupid', 'garbage'] 属于非侮辱类
[Finished in 0.3s]
```

从结果发现，这样写的算法无法进行分类， p_0 和 p_1 的计算结果都是 0，这里显然存在问题，那么需要改进的地方在哪里呢？

利用贝叶斯分类器对文档进行分类时，要计算多个概率的乘积以获得文档属于某个类别的概率，即计算 $P(w_0|1)P(w_1|1)P(w_2|1)$ 。如果其中有一个概率值为 0，那么最后的成绩也为 0。我们拿出上面的截图

```

247 if __name__ == '__main__':
248     postingList, classVec = loadDataSet()
249     myVocabList = createVocabList(postingList)
250     print('myVocabList:\n', myVocabList)
251     trainMat = []
252     for postinDoc in postingList:
253         trainMat.append(setOfWords2Vec(myVocabList, postinDoc))
254     p0V, p1V, pAb = trainNB0(trainMat, classVec)
255     print('p0V:\n', p0V)
256     print('p1V:\n', p1V)
257     print('classVec:\n', classVec)
258     print('pAb:\n', pAb)

```

myVocabList:

['him', 'is', 'not', 'cute', 'love', 'posting', 'garbage', 'flea', 'quit', 'take', 'stop', 'help', 'buying', 'has', 'licks', 'please', 'my', 'dalmation',
'worthless', 'problems', 'to', 'maybe', 'mm', 'park', 'how', 'so', 'stupid', 'food', 'steak', 'ate', 'I', 'dog']

p0V:

0.08333333	0.04166667	0.	0.04166667	0.04166667	0.
0.04166667	0.	0.	0.04166667	0.04166667	0.
0.04166667	0.04166667	0.04166667	0.125	0.04166667	0.
0.04166667	0.04166667	0.	0.04166667	0.	0.04166667
0.04166667	0.	0.	0.04166667	0.04166667	0.04166667
0.04166667					

p0V的倒数第6个概率

p1V:

0.05263158	0.	0.05263158	0.	0.	0.05263158
0.05263158	0.	0.05263158	0.05263158	0.05263158	0.
0.05263158	0.	0.	0.	0.	0.
0.10526316	0.	0.05263158	0.05263158	0.	0.05263158
0.	0.	0.15789474	0.05263158	0.	0.
0.10526316]					

p1V的倒数第6个概率

classVec:

[0, 1, 0, 1, 0, 1]

pAb:

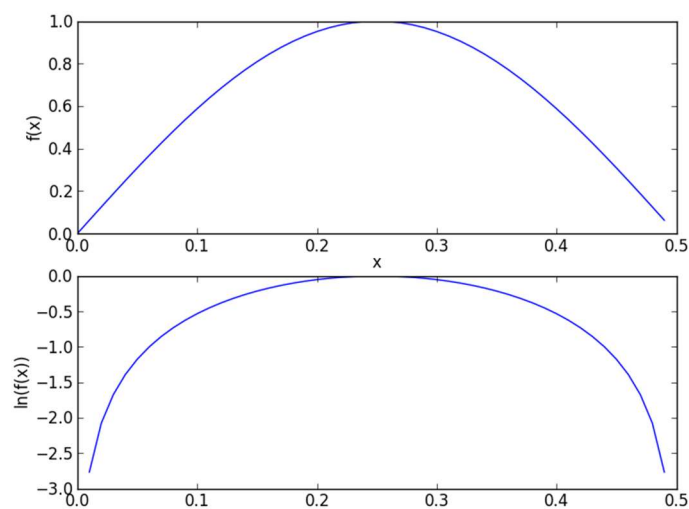
[0.5]

[Epoches in 0.3s]

从上图可以看出，在计算的时候已经出现了概率为 0 的情况。如果新实例文本，包含这种概率为 0 的分词，那么最终的文本属于某个类别的概率也就是 0 了。显然，这样是不合理的，为了降低这种影响，可以将所有词的出现数初始化为 1，并将分母初始化为 2。这种做法就叫做拉普拉斯平滑(Laplace Smoothing)又被称为加 1 平滑，是比较常用的平滑方法，它就是为了解决 0 概率问题。

除此之外，另外一个遇到的问题就是下溢出，这是由于太多很小的数相乘造成的。学过数学的人都知道，两个小数相乘，越乘越小，这样就造成了下溢出。在程序中，在相应小数位置进行四舍五入，计算结果可能就变成 0 了。为了解决

这个问题，对乘积结果取自然对数。通过求对数可以避免下溢出或者浮点数舍入导致的错误。同时，采用自然对数进行处理不会有任何损失。下图给出函数 $f(x)$ 和 $\ln(f(x))$ 的曲线。



检查这两条曲线，就会发现它们在相同区域内同时增加或者减少，并且在相同点上取到极值。它们的取值虽然不同，但不影响最终结果。

继续编程，运行代码，就可以得到如下结果：

```

255 if __name__ == '__main__':
256     postingList, classVec = loadDataSet()
257     myVocabList = createVocabList(postingList)
258     print('myVocabList:\n', myVocabList)
259     trainMat = []
260     for postinDoc in postingList:
261         trainMat.append(setOfWords2Vec(myVocabList, postinDoc))
262     p0V, p1V, pAb = trainNB0(trainMat, classVec)
263     print('p0V:\n', p0V)
264     print('p1V:\n', p1V)
265     print('classVec:\n', classVec)
266     print('pAb:\n', pAb)

myVocabList:
['maybe', 'is', 'to', 'take', 'problems', 'mr', 'not', 'i', 'posting', 'stupid', 'steak', 'garbage', 'help', 'so', 'love', 'please', 'dalmation', 'licks', 'how', 'cute', 'quit',
'flea', 'dog', 'my', 'him', 'buying', 'food', 'worthless', 'park', 'ate', 'has', 'stop']

p0V:
[[-3.25809654 -2.56494936 -2.56494936 -3.25809654 -2.56494936 -2.56494936
-3.25809654 -2.56494936 -3.25809654 -3.25809654 -2.56494936 -3.25809654
-2.56494936 -2.56494936 -2.56494936 -2.56494936 -2.56494936 -2.56494936
-2.56494936 -2.56494936 -3.25809654 -2.56494936 -2.56494936 -1.87188218
-2.15948425 -3.25809654 -3.25809654 -3.25809654 -3.25809654 -2.56494936
-2.56494936 -2.56494936]

p1V:
[[-2.35137526 -3.04452244 -2.35137526 -2.35137526 -3.04452244 -3.04452244
-2.35137526 -3.04452244 -2.35137526 -1.65822888 -3.04452244 -2.35137526
-3.04452244 -3.04452244 -3.04452244 -3.04452244 -3.04452244 -3.04452244
-3.04452244 -3.04452244 -2.35137526 -3.04452244 -1.94591015 -3.04452244
-2.35137526 -2.35137526 -2.35137526 -1.94591015 -2.35137526 -3.04452244
-3.04452244 -2.35137526]

classVec:
[0, 1, 0, 1, 0, 1]

pAb:
0.5

[Finished in 2.9s]

```

这样我们得到的结果就没有问题了，不存在 0 概率。这样，我们的朴素贝叶斯分类器就改进完毕了。总代码编写如下：

```

# -*- coding: UTF-8 -*-

import numpy as np
from functools import reduce

"""
函数说明：创建实验样本

Parameters:
无

```

Returns:

postingList - 实验样本切分的词条
classVec - 类别标签向量

```
"""
def loadDataSet():
    postingList=[['my', 'dog', 'has', 'flea', 'problems', 'help',
'please'],
                  #切分的词条
                  ['maybe', 'not', 'take', 'him', 'to', 'dog', 'park',
'stupid'],
                  ['my', 'dalmation', 'is', 'so', 'cute', 'I', 'love',
'him'],
                  ['stop', 'posting', 'stupid', 'worthless', 'garbage'],
                  ['mr', 'licks', 'ate', 'my', 'steak', 'how', 'to',
'stop', 'him'],
                  ['quit', 'buying', 'worthless', 'dog', 'food',
'stupid']]
    classVec = [0,1,0,1,0,1]
    #类别标签向量, 1 代表侮辱性词汇, 0 代表不是
    return postingList,classVec
#返回实验样本切分的词条和类别标签向量
```

函数说明:将切分的实验样本词条整理成不重复的词条列表, 也就是词汇表

Parameters:

dataSet - 整理的样本数据集

Returns:

vocabSet - 返回不重复的词条列表, 也就是词汇表

```
"""
def createVocabList(dataSet):
    vocabSet = set([])
    #创建一个空的不重复列表
    for document in dataSet:
        vocabSet = vocabSet | set(document) #取并集
    return list(vocabSet)
```

函数说明:根据 vocabList 词汇表, 将 inputSet 向量化, 向量的每个元素为 1 或 0

Parameters:

vocabList - createVocabList 返回的列表

inputSet - 切分的词条列表

Returns:

```

returnVec - 文档向量,词集模型

"""
def setOfWords2Vec(vocabList, inputSet):
    returnVec = [0] * len(vocabList)           #创建一个
    其中所含元素都为0的向量
    for word in inputSet:                       #遍历每个词条
        if word in vocabList:                   #如果词条存在于
            词汇表中,则置1
            returnVec[vocabList.index(word)] = 1
        else: print("the word: %s is not in my Vocabulary!" % word)
    return returnVec                           #返回文档向量

"""
函数说明:朴素贝叶斯分类器训练函数

Parameters:
    trainMatrix - 训练文档矩阵,即 setOfWords2Vec 返回的 returnVec 构成的
    矩阵
    trainCategory - 训练类别标签向量,即 loadDataSet 返回的 classVec
Returns:
    p0Vect - 侮辱类的条件概率数组
    p1Vect - 非侮辱类的条件概率数组
    pAbusive - 文档属于侮辱类的概率

"""
def trainNB0(trainMatrix,trainCategory):
    numTrainDocs = len(trainMatrix)           #计算训练
    文档数目
    numWords = len(trainMatrix[0])            #计算每篇
    文档的词条数
    pAbusive = sum(trainCategory)/float(numTrainDocs) #文档属
    于侮辱类的概率
    p0Num = np.ones(numWords); p1Num = np.ones(numWords) #创建
    numpy.ones 数组,词条出现数初始化为1,拉普拉斯平滑
    p0Denom = 2.0; p1Denom = 2.0             #分母初始化为
    2,拉普拉斯平滑
    for i in range(numTrainDocs):
        if trainCategory[i] == 1:             #统计属于侮
            辱类的条件概率所需的数据,即  $P(w_0|1), P(w_1|1), P(w_2|1) \dots$ 
            p1Num += trainMatrix[i]
            p1Denom += sum(trainMatrix[i])
        else:                                  #统计属于非侮辱

```

类的条件概率所需的数据, 即 $P(w_0|0), P(w_1|0), P(w_2|0) \dots$

```
        p0Num += trainMatrix[i]
        p0Denom += sum(trainMatrix[i])
        p1Vect = np.log(p1Num/p1Denom)          #取对数,
防止下溢出
        p0Vect = np.log(p0Num/p0Denom)
        return p0Vect,p1Vect,pAbusive          #返回属于侮辱
类的条件概率数组, 属于非侮辱类的条件概率数组, 文档属于侮辱类的概率
```

"""

函数说明:朴素贝叶斯分类器分类函数

Parameters:

vec2Classify - 待分类的词条数组
p0Vec - 侮辱类的条件概率数组
p1Vec -非侮辱类的条件概率数组
pClass1 - 文档属于侮辱类的概率

Returns:

0 - 属于非侮辱类
1 - 属于侮辱类

"""

```
def classifyNB(vec2Classify, p0Vec, p1Vec, pClass1):
    p1 = sum(vec2Classify * p1Vec) + np.log(pClass1)          #对应元素
    相乘。logA * B = logA + logB, 所以这里加上 log(pClass1)
    p0 = sum(vec2Classify * p0Vec) + np.log(1.0 - pClass1)
    if p1 > p0:
        return 1
    else:
        return 0
```

"""

函数说明:测试朴素贝叶斯分类器

Parameters:

无

Returns:

无

"""

```
def testingNB():
    listOPosts,listClasses = loadDataSet()          #创建
    实验样本
    myVocabList = createVocabList(listOPosts)      #创
```

建词汇表

```
trainMat=[]
for postinDoc in listOPosts:
    trainMat.append(setOfWords2Vec(myVocabList, postinDoc))
#将实验样本向量化
p0V,p1V,pAb =
trainNB0(np.array(trainMat),np.array(listClasses)) #训练朴素贝
叶斯分类器
testEntry = ['love', 'my', 'dalmation'] #
测试样本1
thisDoc = np.array(setOfWords2Vec(myVocabList, testEntry))
#测试样本向量化
if classifyNB(thisDoc,p0V,p1V,pAb):
    print(testEntry,'属于侮辱类') #执行分类
并打印分类结果
else:
    print(testEntry,'属于非侮辱类') #执行分类
并打印分类结果
testEntry = ['stupid', 'garbage'] #测试样
本2

thisDoc = np.array(setOfWords2Vec(myVocabList, testEntry))
#测试样本向量化
if classifyNB(thisDoc,p0V,p1V,pAb):
    print(testEntry,'属于侮辱类') #执行分类
并打印分类结果
else:
    print(testEntry,'属于非侮辱类') #执行分类
并打印分类结果

if __name__ == '__main__':
    postingList, classVec = loadDataSet()
    myVocabList = createVocabList(postingList)
    print('myVocabList:\n', myVocabList)
    trainMat = []
    for postinDoc in postingList:
        trainMat.append(setOfWords2Vec(myVocabList, postinDoc))
    p0V, p1V, pAb = trainNB0(trainMat, classVec)
    print('p0V:\n', p0V)
    print('p1V:\n', p1V)
    print('classVec:\n', classVec)
    print('pAb:\n', pAb)
    testingNB()
```

编程完毕，运行结果如下：

```
set(words,vec(vocabList, inputSet))
trainNB0(trainMatrix, trainCategory)
vocabSet = 返回不重复的词条列表，也就是词汇表
classifyNB(vec2Classify, p0Vec, p1Vec, pClass)
loadDataSet()
testinoNB0()

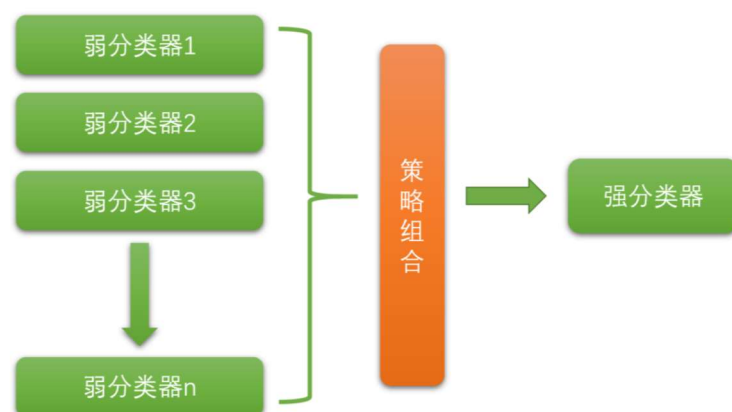
运行: bayes
D:\ProgramData\Anaconda3\python.exe "E:/机器学习/机器学习实验指导书/代码/P4-Naive Bayes-朴素贝叶斯/bayes.py"
myVocabList:
['maybe', 'dog', 'has', 'buying', 'licks', 'love', 'please', 'my', 'flea', 'is', 'park', 'not', 'garbage', 'stop']
p0V:
[-3.25809654 -2.56494936 -2.56494936 -3.25809654 -2.56494936 -2.56494936
-2.56494936 -1.87180218 -2.56494936 -2.56494936 -3.25809654 -3.25809654
-3.25809654 -2.56494936 -3.25809654 -2.56494936 -3.25809654 -2.56494936
-3.25809654 -2.56494936 -2.56494936 -3.25809654 -2.15948425 -3.25809654
-2.56494936 -2.56494936 -2.56494936 -2.56494936 -3.25809654 -2.56494936
-2.56494936 -2.56494936]
p1V:
[-2.35137526 -1.94591015 -3.04452244 -2.35137526 -3.04452244 -3.04452244
-3.04452244 -3.04452244 -3.04452244 -3.04452244 -2.35137526 -2.35137526
-2.35137526 -2.35137526 -2.35137526 -3.04452244 -2.35137526 -3.04452244
-1.65822808 -3.04452244 -3.04452244 -2.35137526 -2.35137526 -2.35137526
-3.04452244 -3.04452244 -3.04452244 -3.04452244 -1.94591015 -2.35137526
-3.04452244 -3.04452244]
['love', 'my', 'dalmation'] 属于非侮辱类
['stupid', 'garbage'] 属于侮辱类
```

(二) 集成学习分类实验

前面我们已经学习了 6 种不同的分类器，它们各有优缺点。所以可以将不同的分类器组合起来，而这种组合结果则被成为集成方法(ensemble method)或者元算法(meta-algorithm)。使用集成方法时会有多种形式：可以是不同算法的集成，也可以是同一种算法在不同设置下的集成，还可以是数据集不同分配给不同分类器之后的集成。

1、集成方法

集成学习 (Ensemble learning) 就是将若干个弱分类器通过一定的策略组合之后产生一个强分类器。弱分类器 (Weak Classifier) 指的就是那些分类准确率只比随机猜测略好一点的分类器，而强分类器 (Strong Classifier) 的分类准确率会高很多。这里的"强"&"弱"是相对的。弱分类器也称为“基分类器”。

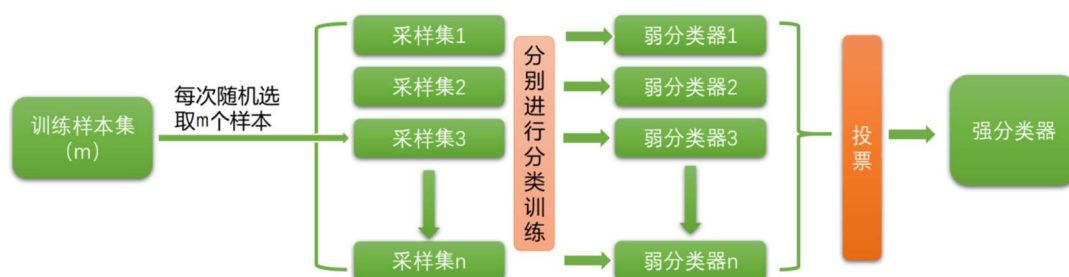


集成方法主要包括 Bagging 和 Boosting 两种方法，Bagging 和 Boosting 都是将已有的分类或回归算法通过一定方式组合起来，形成一个性能更加强大的分类器，更准确的说这是一种分类算法的组装方法，即将弱分类器组装成强分类器的方法。

2、Bagging

自举汇聚法（bootstrap aggregating），也称为 bagging 方法。Bagging 对训练数据采用自举采样（bootstrap sampling），即有放回地采样数据，主要思想：

- 从原始样训练本集中抽取采练集。每轮从原始样本集中随机抽取 m 个训练样本（在训练集中，有些样本可能被多次抽取到，而有些样本可能一次都没有被抽中）。共进行 n 轮抽取，得到 n 个采练集。（ n 个训练集之间是相互独立的）；
- 每次使用一个训练集一个弱分类器得到一个模型， n 个训练集共得到 n 个模型。（注：这里并没有具体的分类算法或回归方法，我们可以根据具体问题采用不同的分类或回归方法，如决策树、逻辑回归等）；
- 对分类问题：将上步得到的 n 个模型采用投票的方式得到分类结果；对回归问题，计算上述模型的均值作为最后的结果。（所有弱分类器的重要性相同）。



3、Boosting

Boosting 是一种与 Bagging 很类似的技术。Boosting 的思路则是采用重赋权（re-weighting）法迭代地训练基分类器，主要思想：

- 每一轮的训练数据样本赋予一个权重，并且每一轮样本的权值分布依赖上一轮的分类结果。
- 基分类器之间采用序列式的线性加权方式进行组合。



4、Bagging、Boosting 二者之间的区别

样本选择上：

Bagging: 训练集是在原始集中有放回选取的，从原始集中选出的各轮训练集之间是独立的。

Boosting: 每一轮的训练集不变，只是训练集中每个样例在分类器中的权重发生变化。而权值是根据上一轮的分类结果进行调整。

样例权重：

Bagging: 使用均匀取样，每个样例的权重相等。

Boosting: 根据错误率不断调整样例的权值，错误率越大则权重越大。

预测函数：

Bagging: 所有预测函数的权重相等。

Boosting: 每个弱分类器都有相应的权重，对于分类误差小的分类器会有更大的权重。

并行计算：

Bagging: 各个预测函数可以并行生成。

Boosting: 各个预测函数只能顺序生成，因为后一个模型参数需要前一轮模型的结果。

这两种方法都是把若干个分类器整合为一个分类器的方法，只是整合的方式不一样，最终得到不一样的效果，将不同的分类算法套入到此类算法框架中一定程度上会提高了原单一分类器的分类效果，但是也增大了计算量。

下面是将决策树与这些算法框架进行结合所得到的新的算法：

Bagging + 决策树 = 随机森林

AdaBoost + 决策树 = 提升树

Gradient Boosting + 决策树 = GBDT

集成方法众多，本章主要关注 **Boosting** 方法中的一种最流行的版本，即 **AdaBoost**。

5、AdaBoost

AdaBoost 算法是基于 **Boosting** 思想的机器学习算法，**AdaBoost** 是 **adaptive boosting**（自适应 **boosting**）的缩写，其运行过程如下：

（1）计算样本权重

训练数据中的每个样本，赋予其权重，即样本权重，用向量 D 表示，这些权重都初始化成相等值。假设有 n 个样本的训练集：

$$\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

设定每个样本的权重都是相等的，即 $1/n$ 。

(2) 计算错误率

利用第一个弱学习算法 h_1 对其进行学习,学习完成后进行错误率 ε 的统计:

$$\varepsilon = \frac{\text{未正确分类的样本数目}}{\text{所有样本数目}}$$

(3) 计算弱学习算法权重

弱学习算法也有一个权重,用向量 α 表示,利用错误率计算权重 α :

$$\alpha = \frac{1}{2} \ln \left(\frac{1-\varepsilon}{\varepsilon} \right)$$

(4) 更新样本权重

根据上一次训练结果,调整权重值(上一次分对的权重降低,分错的权重增加)

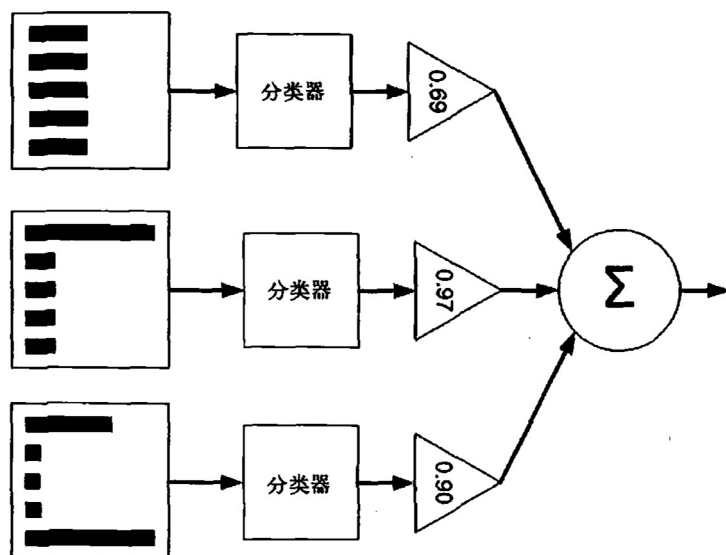
如果第 i 个样本被正确分类,则该样本权重更改为:

$$D_i^{(t+1)} = \frac{D_i^{(t)} e^{-\alpha}}{\text{Sum}(D)}$$

如果第 i 个样本被分错,则该样本权重更改为:

$$D_i^{(t+1)} = \frac{D_i^{(t)} e^{\alpha}}{\text{Sum}(D)}$$

之后,在同一数据集上再一次训练弱分类器,然后循环上述过程,直到训练错误率为 0,或者弱分类器的数目达到指定值。具体过程如下所示:



6、基于单层决策树构建弱分类器

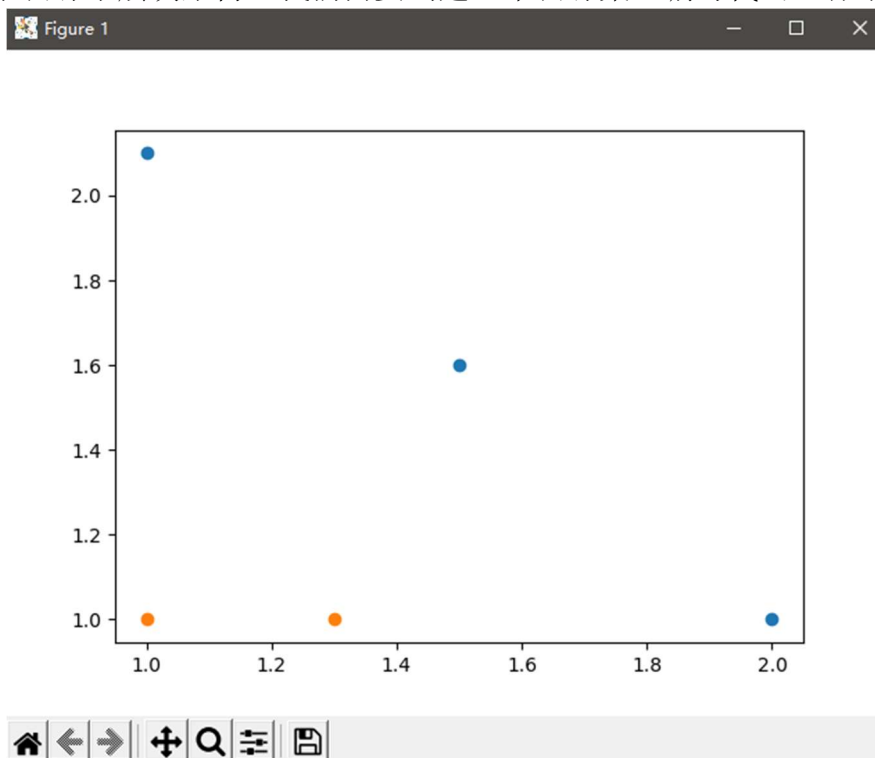
对给定数据集使用 AdaBoost 算法建立弱分类器,并保存样本的权重。

弱分类器使用单层决策树,通过给定的阈值,进行分类。

使用 AdaBoost 提升分类器性能

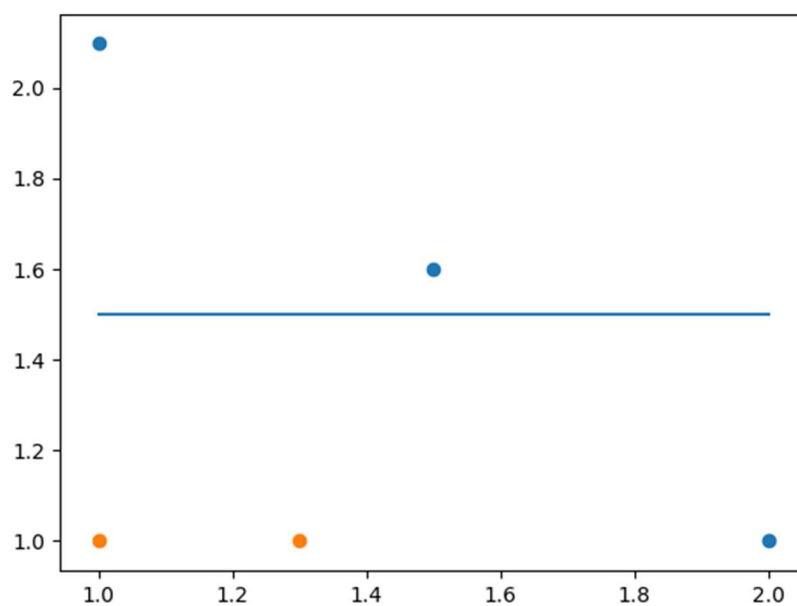
(1) 数据集可视化

为了训练单层决策树，我们需要创建一个训练集，编写代码运行结果如下：



可以看到，如果想要试着从某个坐标轴上选择一个值（即选择一条与坐标轴平行的直线）来将所有的蓝色圆点和橘色圆点分开，这显然是不可能的。这就是单层决策树难以处理的一个著名问题。通过使用多颗单层决策树，我们可以构建出一个能够对该数据集完全正确分类的分类器。

我们设置一个分类阈值，比如我横向切分，用的是特征 2，如下图所示：



蓝横线上边的是一个类别，蓝横线下边是一个类别。显然，此时有一个蓝点分类错误，计算此时的分类误差，误差为 $1/5 = 0.2$ 。这个横线与坐标轴的 y 轴的交点，就是我们设置的阈值，通过不断改变阈值的大小，找到使单层决策树的分类误差最小的阈值。同理，用特征 1, 竖线也是如此，找到最佳分类的阈值，就找到了最佳单层决策树，编写代码运行结果如下：

```

133 if __name__ == '__main__':
134     dataArr, classLabels = loadSimpData()
135     D = np.mat(np.ones((5, 1)) / 5)
136     bestStump, minError, bestClasEst = buildStump(dataArr, classLabels, D)
137     print('bestStump:\n', bestStump)
138     print('minError:\n', minError)
139     print('bestClasEst:\n', bestClasEst)

split: dim 1, thresh 1.44, thresh inequal: gt, the weighted error is 0.800
split: dim 1, thresh 1.55, thresh inequal: lt, the weighted error is 0.200
split: dim 1, thresh 1.55, thresh inequal: gt, the weighted error is 0.800
split: dim 1, thresh 1.66, thresh inequal: lt, the weighted error is 0.400
split: dim 1, thresh 1.66, thresh inequal: gt, the weighted error is 0.600
split: dim 1, thresh 1.77, thresh inequal: lt, the weighted error is 0.400
split: dim 1, thresh 1.77, thresh inequal: gt, the weighted error is 0.600
split: dim 1, thresh 1.88, thresh inequal: lt, the weighted error is 0.400
split: dim 1, thresh 1.88, thresh inequal: gt, the weighted error is 0.600
split: dim 1, thresh 1.99, thresh inequal: lt, the weighted error is 0.400
split: dim 1, thresh 1.99, thresh inequal: gt, the weighted error is 0.600
split: dim 1, thresh 2.10, thresh inequal: lt, the weighted error is 0.600
split: dim 1, thresh 2.10, thresh inequal: gt, the weighted error is 0.400
bestStump:
{'thresh': 1.3, 'ineq': 'lt', 'dim': 0}
minError:
[[ 0.2]]
bestClasEst:
[[-1.]
 [ 1.]
 [-1.]
 [-1.]
 [ 1.]]
[Finished in 0.6s]

```

代码不难理解，就是通过遍历，改变不同的阈值，计算最终的分类误差，找到分类误差最小的分类方式，即为我们要找的最佳单层决策树。这里 lt 表示 less than，表示分类方式，对于小于阈值的样本点赋值为-1，gt 表示 greater than，也是表示分类方式，对于大于阈值的样本点赋值为-1。经过遍历，我们找到，训练好的最佳单层决策树的最小分类误差为 0.2，就是对于该数据集，无论用什么样的单层决策树，分类误差最小就是 0.2。这就是我们训练好的弱分类器。接下来，使用 AdaBoost 算法提升分类器性能，将分类误差缩短到 0，看下 AdaBoost 算法是如何实现的。

(2) 使用 AdaBoost 提升分类器性能

根据之前介绍的 AdaBoost 算法实现过程，使用 AdaBoost 算法提升分类器性能，编写代码运行结果如下：

```

split: dim 1, thresh 1.11, thresh inequal: gt, the weighted error is 0.500
split: dim 1, thresh 1.22, thresh inequal: lt, the weighted error is 0.500
split: dim 1, thresh 1.22, thresh inequal: gt, the weighted error is 0.500
split: dim 1, thresh 1.33, thresh inequal: lt, the weighted error is 0.500
split: dim 1, thresh 1.33, thresh inequal: gt, the weighted error is 0.500
split: dim 1, thresh 1.44, thresh inequal: lt, the weighted error is 0.500
split: dim 1, thresh 1.44, thresh inequal: gt, the weighted error is 0.500
split: dim 1, thresh 1.55, thresh inequal: lt, the weighted error is 0.500
split: dim 1, thresh 1.55, thresh inequal: gt, the weighted error is 0.500
split: dim 1, thresh 1.66, thresh inequal: lt, the weighted error is 0.571
split: dim 1, thresh 1.66, thresh inequal: gt, the weighted error is 0.429
split: dim 1, thresh 1.77, thresh inequal: lt, the weighted error is 0.571
split: dim 1, thresh 1.77, thresh inequal: gt, the weighted error is 0.429
split: dim 1, thresh 1.88, thresh inequal: lt, the weighted error is 0.571
split: dim 1, thresh 1.88, thresh inequal: gt, the weighted error is 0.429
split: dim 1, thresh 1.99, thresh inequal: lt, the weighted error is 0.571
split: dim 1, thresh 1.99, thresh inequal: gt, the weighted error is 0.429
split: dim 1, thresh 2.10, thresh inequal: lt, the weighted error is 0.857
split: dim 1, thresh 2.10, thresh inequal: gt, the weighted error is 0.143
D: [[ 0.28571429  0.07142857  0.07142857  0.07142857  0.5        ]]
classEst: [[ 1.  1.  1.  1.  1.]]
aggClassEst: [[ 1.17568763  2.56198199 -0.77022252 -0.77022252  0.61607184]]
total error: 0.0
[{'ineq': 'lt', 'dim': 0, 'alpha': 0.6931471805599453, 'thresh': 1.3}, {'ineq': 'lt', 'dim':
[[ 1.17568763]
[ 2.56198199]
[-0.77022252]
[-0.77022252]
[ 0.61607184]]

```

在第一轮迭代中，D 中的所有值都相等。于是，只有第一个数据点被错分了。因此在第二轮迭代中，D 向量给第一个数据点 0.5 的权重。这就可以通过变量 `aggClassEst` 的符号来了解总的类别。第二次迭代之后，我们就会发现第一个数据点已经正确分类了，但此时最后一个数据点却是错分了。D 向量中的最后一个元素变为 0.5，而 D 向量中的其他值都变得非常小。最后，第三次迭代之后 `aggClassEst` 所有值的符号和真是类别标签都完全吻合，那么训练错误率为 0，程序终止运行。

最后训练结果包含了三个弱分类器，其中包含了分类所需要的所有信息。一共迭代了 3 次，所以训练了 3 个弱分类器构成一个使用 AdaBoost 算法优化过的分类器，分类器的错误率为 0。

一旦拥有了多个弱分类器以及其对应的 `alpha` 值，进行测试就变得相当容易了。编写代码如下：

```

# -*-coding:utf-8 -*-
import numpy as np
import matplotlib.pyplot as plt

def loadSimpData():
    """
    创建单层决策树的数据集
    Parameters:
        无
    Returns:

```

```

        dataMat - 数据矩阵
        classLabels - 数据标签
    """
    datMat = np.matrix([[ 1. ,  2.1],
        [ 1.5,  1.6],
        [ 1.3,  1. ],
        [ 1. ,  1. ],
        [ 2. ,  1. ]])
    classLabels = [1.0, 1.0, -1.0, -1.0, 1.0]
    return datMat,classLabels

def showDataSet(dataMat, labelMat):
    """
    数据可视化
    Parameters:
        dataMat - 数据矩阵
        labelMat - 数据标签
    Returns:
        无
    """
    data_plus = [] # 正样本
    data_minus = [] # 负样本
    for i in range(len(dataMat)):
        if labelMat[i] > 0:
            data_plus.append(dataMat[i])
        else:
            data_minus.append(dataMat[i])
    data_plus_np = np.array(data_plus) # 转
    # 换为 numpy 矩阵
    data_minus_np = np.array(data_minus) #
    # 转换为 numpy 矩阵
    plt.scatter(np.transpose(data_plus_np)[0],
np.transpose(data_plus_np)[1]) # 正样本散点图
    plt.scatter(np.transpose(data_minus_np)[0],
np.transpose(data_minus_np)[1]) # 负样本散点图
    plt.show()

def stumpClassify(dataMatrix, dimen, threshVal, threshIneq):
    """
    单层决策树分类函数
    Parameters:
        dataMatrix - 数据矩阵
        dimen - 第 dimen 列, 也就是第几个特征
        threshVal - 阈值
    """

```

```

    threshIneq - 标志
Returns:
    retArray - 分类结果
"""
    retArray = np.ones((np.shape(dataMatrix)[0],1)) #初始
    化 retArray 为1
    if threshIneq == 'lt':
        retArray[dataMatrix[:,dimen] <= threshVal] = -1.0 #如果小于
        阈值,则赋值为-1
    else:
        retArray[dataMatrix[:,dimen] > threshVal] = -1.0 #如果大于
        阈值,则赋值为-1
    return retArray

def buildStump(dataArr,classLabels,D):
    """
    找到数据集上最佳的单层决策树
    Parameters:
        dataArr - 数据矩阵
        classLabels - 数据标签
        D - 样本权重
    Returns:
        bestStump - 最佳单层决策树信息
        minError - 最小误差
        bestClasEst - 最佳的分类结果
    """
    dataMatrix = np.mat(dataArr); labelMat = np.mat(classLabels).T
    m,n = np.shape(dataMatrix)
    numSteps = 10.0; bestStump = {}; bestClasEst =
    np.mat(np.zeros((m,1)))
    minError = float('inf') #最小
    误差初始化为正无穷大
    for i in range(n): #遍历所有特
        征
        rangeMin = dataMatrix[:,i].min(); rangeMax =
        dataMatrix[:,i].max() #找到特征中最小的值和最大值
        stepSize = (rangeMax - rangeMin) / numSteps
        #计算步长
        for j in range(-1, int(numSteps) + 1):
            for inequal in ['lt', 'gt']: #大于
                和小于的情况, 均遍历。lt:less than, gt:greater than
                threshVal = (rangeMin + float(j) * stepSize)
                #计算阈值

```

```

        predictedVals = stumpClassify(dataMatrix, i, threshVal,
inequal) # 计算分类结果
        errArr = np.mat(np.ones((m,1))) # 初
始化误差矩阵
        errArr[predictedVals == labelMat] = 0 #
分类正确的,赋值为0
        weightedError = D.T * errArr # 计算
误差
        print("split: dim %d, thresh %.2f, thresh inequal: %s,
the weighted error is %.3f" % (i, threshVal, inequal,
weightedError))
        if weightedError < minError: # 找到
误差最小的分类方式
            minError = weightedError
            bestClasEst = predictedVals.copy()
            bestStump['dim'] = i
            bestStump['thresh'] = threshVal
            bestStump['ineq'] = inequal
            return bestStump, minError, bestClasEst

def adaBoostTrainDS(dataArr, classLabels, numIt = 40):
    """
    使用 AdaBoost 算法提升弱分类器性能
    Parameters:
        dataArr - 数据矩阵
        classLabels - 数据标签
        numIt - 最大迭代次数
    Returns:
        weakClassArr - 训练好的分类器
        aggClassEst - 类别估计累计值
    """
    weakClassArr = []
    m = np.shape(dataArr)[0]
    D = np.mat(np.ones((m, 1)) / m) # 初
始化权重
    aggClassEst = np.mat(np.zeros((m,1)))
    for i in range(numIt):
        bestStump, error, classEst = buildStump(dataArr,
classLabels, D) # 构建单层决策树
        print("D:",D.T)
        alpha = float(0.5 * np.log((1.0 - error) / max(error, 1e-
16))) # 计算弱学习算法权重 alpha,使 error 不等于 0,因为分母不能为 0
        bestStump['alpha'] = alpha # 存储弱学
习算法权重

```

```

        weakClassArr.append(bestStump)                                #存
#单层决策树
        print("classEst: ", classEst.T)
        expon = np.multiply(-1 * alpha * np.mat(classLabels).T,
classEst) #计算e的指数项
        D = np.multiply(D, np.exp(expon))
        D = D / D.sum()                                            #根据样本权重公式,更新样本权重
        #计算AdaBoost误差,当误差为0的时候,退出循环
        aggClassEst += alpha * classEst                            #计算类别估计累计值
        print("aggClassEst: ", aggClassEst.T)
        aggErrors = np.multiply(np.sign(aggClassEst) !=
np.mat(classLabels).T, np.ones((m,1))) #计算误差
        errorRate = aggErrors.sum() / m
        print("total error: ", errorRate)
        if errorRate == 0.0: break                                  #误差为0,退出循环
        return weakClassArr, aggClassEst

def adaClassify(datToClass,classifierArr):
    """
    AdaBoost 分类函数
    Parameters:
        datToClass - 待分类样例
        classifierArr - 训练好的分类器
    Returns:
        分类结果
    """
    dataMatrix = np.mat(datToClass)
    m = np.shape(dataMatrix)[0]
    aggClassEst = np.mat(np.zeros((m,1)))
    for i in range(len(classifierArr)):                             #
#遍历所有分类器,进行分类
        classEst = stumpClassify(dataMatrix,
classifierArr[i]['dim'], classifierArr[i]['thresh'],
classifierArr[i]['ineq'])
        aggClassEst += classifierArr[i]['alpha'] * classEst
        print(aggClassEst)
    return np.sign(aggClassEst)

if __name__ == '__main__':
    dataArr,classLabels = loadSimpData()

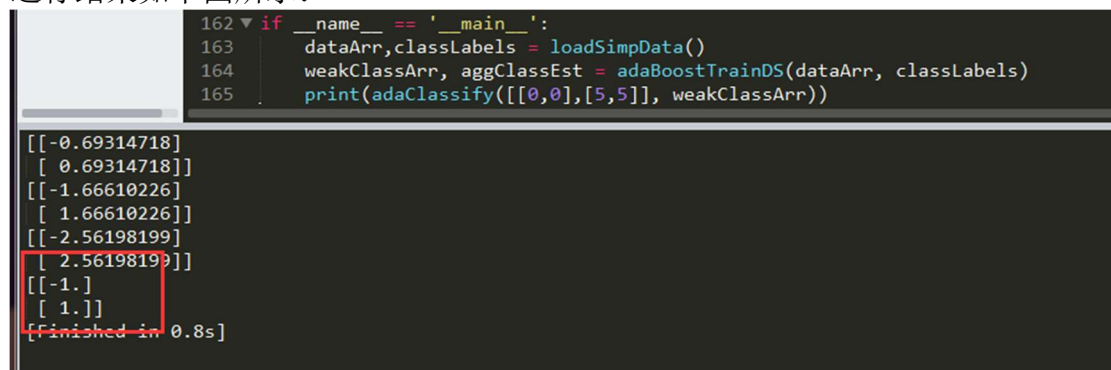
```

```

showDataSet(dataArr, classLabels)
weakClassArr, aggClassEst = adaBoostTrainDS(dataArr,
classLabels)
print(adaClassify([[0,0],[5,5]], weakClassArr))

```

运行结果如下图所示：



```

162 if __name__ == '__main__':
163     dataArr, classLabels = loadSimpData()
164     weakClassArr, aggClassEst = adaBoostTrainDS(dataArr, classLabels)
165     print(adaClassify([[0,0],[5,5]], weakClassArr))

```

```

[[-0.69314718]
 [ 0.69314718]]
[[-1.66610226]
 [ 1.66610226]]
[[-2.56198199]
 [ 2.56198199]]
[[-1.]
 [ 1.]]
[finished in 0.8s]

```

代码很简单，在之前代码的基础上，添加 `adaClassify()` 函数，该函数遍历所有训练得到的弱分类器，利用单层决策树，输出的类别估计值乘以该单层决策树的分类器权重 `alpha`，然后累加到 `aggClassEst` 上，最后通过 `sign` 函数最终的结果。可以看到，分类没有问题，(5,5)属于正类，(0,0)属于负类。

四：实验要求

- 1、根据实验讲义完成相应实验内容，对比实验结果，调整参数，保证结果的正确性；
- 2、记录实验过程，撰写电子版实验报告，注意每张结果图片上需注明同学们的“学号+姓名”；
- 3、完成电子版实验报告后，调整报告格式内容，不限于字体、字号、标题、段落、图片大小等等，保证报告结果正确且美观，检查无误在学习通平台提交电子版实验报告文件，提交前修改电子版文件名称为“学号+姓名+实验 4”；
- 4、打印纸质版实验报告进行保存，所有实验结束后整体装订成册，交给各班学习委员。